

1. Logika cyfrowa

Układ	Formuła / sygnatura	Opis
Sumatory		
Półsumator	$s = x \oplus y$ $c = x \& y$	Dodaje dwa bity. s to suma, c to bit przeniesienia (carry).
Pełny sumator (FA)	$s = x \oplus y \oplus ci$ $co = x \& y \mid ci \& (x \oplus y)$	Jak półsumator, ale przyjmuje też przeniesienie wejściowe ci .
Sumator n-bitowy	$n \times FA$	Kaskadowo wyjście C_i trafia na wejście kolejnego FA. Ostatnie co to Carry Out całości.
Układy kombinacyjne		
Dekoder	$n \rightarrow 2^n$	Wejście koduje liczbę k . Na wyjściu zapalony jest bit k .
Multiplekser	$2^n + n \rightarrow 1$	2^n bitów danych + n bitów sterujących S . Na wyjściu zapalony S -ty bit danych.
ALU	$A, B, f \rightarrow C$	Dekoder na bitach f wybiera operację np. $A + B$, multipleksowanie wyników bramkami AND/OR.
Układy sekwencyjne (pamięć)		
Przerzutnik S-R	S - set, R - reset	Przechowuje 1 bit (Q). S=1 ustawia $Q = 1$, R=1 zeruje, S=R=0 trzyma stan. Dwa sprzężone NOR -y.
Sterowany poziomem	aktywny gdy CLK = 1	Wejścia S / R zAND-owane z zegarem.
Sterowany zboczem	zbocze 1 $\rightarrow$ 0	Dwa przerzutniki poziomowe w kaskadzie. Stan przepisuje się w momencie zmiany zegara.

2. Typy danych

Typ	char	short	unsigned	int	long	float	double	void*
x86-64	1	2	4	4	8	4	8	8

3. Rozszerzanie, obcinanie i przesunięcia

$x \ll k$	$x \cdot 2^k$ (sgn./uns.)
$u \gg k$	$\lfloor u/2^k \rfloor$ - logiczne (zera)
$x \gg k$	arytmetyczne (kopia MSB), zaokr. do $-\infty$
$x/2^k$ do 0	$(x + (1 \ll (k-1)) \gg k$
Strength reduction	$x * 24 \rightarrow (x \ll 5) - (x \ll 3)$

4. Operacje i endianness

Negacja U2

`-x = ~x+1 ; -TMin=TMin , -0=0`

Wykr. nadmiaru (`s=x+y`)

`((s^x)&(s^y))>>(w-1)`

Maska / abs

`m=x>>31; abs=(x^m)-m`

`c ? a : b`

`b ^ (-c & (a ^ b))`

promocja w C:

`unsigned` i `int` w jednym wyrażeniu/porównaniu

`< > ==`) → `int` promowany do `unsigned` (bity bez zmian, `-1` → `Max`).

Schemat	Najniższy adres	0x0123
Big Endian	MSB	[01][23]
Little Endian	LSB	[23][01]

5. Zmiennoprzecinkowe (IEEE 754)

$V = (-1)^s \times M \times 2^E$ Bias = $2^{k-1} - 1$

Format	bity	s	exp (k)	frac (n)
float	32	1	8	23
double	64	1	11	52

Kategorie wartości

Kategoria	exp	Własności
Znormalizowane	$!= 0 \dots 0$ $!= 1 \dots 1$	$E = \text{exp} - \text{Bias}$. $M = 1.\text{frac}$. Najgęściej ułożone wokół 0.
Zdenormaliz.	$== 0 \dots 0$	$E = 1 - \text{Bias}$. $M = 0.\text{frac}$. Reprezentują ± 0.0 (frac = 0) i liczby bardzo bliskie zera.
Specjalne	$== 1 \dots 1$	$\text{frac} == 0 \rightarrow \pm \infty$ (nadmiar, dzielenie przez 0). $\text{frac} != 0 \rightarrow \text{NaN}$ (np. $\sqrt{-1}$, $\infty - \infty$).

Zaokrąglenie GRS i Round-to-Even

Domyślny tryb to **Round-to-Even** (zapobiega błędowi statycznemu przy sumowaniu).

... x x G ! R S S S ... G - ostatni zachowany, R - pierwszy usunięty, S - 0R reszty		
Warunek	Ułamek	Akcja
R=0	< 0.5	W dół (odcięcie)
R=1, S=1	> 0.5	W górę (+1 do G)
R=1, S=0	$= 0.5$	Do parzystej: w górę gdy G=1 , w dół gdy G=0

Własności matematyczne i rzutowanie

Brak łączności	$(a + b) + c \neq a + (b + c)$ - zaokrąglenia
Brak rozdzielności	$a(b + c) \neq ab + ac$
int $\rightarrow$ float	możliwa utrata precyzji (zaokrągła)
int $\rightarrow$ double	bezstratne ($52 > 32$ bity)
float/double $\rightarrow$ int	obcina do 0; poza zakresem lub NaN $\rightarrow$ TMin (0x80000000)

6. Rejestry x86_64

<code>%rax</code>	<code>%eax</code>	<code>%ax</code> <code>%ah</code> <code>%al</code>
<code>%rbx</code>	<code>%ebx</code>	<code>%bx</code> <code>%bh</code> <code>%bl</code>

<code>%rcx</code>	<code>%ecx</code>	<code>%cx</code> <code>%ch</code> <code>%cl</code>
<code>%rdx</code>	<code>%edx</code>	<code>%dx</code> <code>%dh</code> <code>%dl</code>

<code>%rsi</code>	<code>%esi</code>	<code>%si</code> <code>%sil</code>
<code>%rdi</code>	<code>%edi</code>	<code>%di</code> <code>%dil</code>

<code>%rbp</code>	<code>%ebp</code>	<code>%bp</code> <code>%bpl</code>
<code>%rsp</code>	<code>%esp</code>	<code>%sp</code> <code>%spl</code>

<code>%r8</code>	<code>%r8d</code>	<code>%r8w</code> <code>%r8b</code>
<code>%r9</code>	<code>%r9d</code>	<code>%r9w</code> <code>%r9b</code>

Uwaga: Rejestry od `%r10` do `%r15` używają schematu:
`%r10` , `%r10d` , `%r10w` , `%r10b` .

Podział ról rejestrów

<code>%rax</code>	Akumulator. Główny rejestr arytmetyczny. Przechowuje wartość zwracaną .
<code>%rdi</code> , <code>%rsi</code> , <code>%rdx</code> , <code>%rcx</code> , <code>%r8</code> , <code>%r9</code>	Kolejno: od 1. do 6. argumentu funkcji . Caller-saved.
<code>%rsp</code>	Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.
<code>%rbp</code>	Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.
<code>%rbx</code> , <code>%r12</code> - <code>%r15</code>	Ogólnego przeznaczenia. Wszystkie callee-saved .
<code>%rip</code>	Wskaźnik instrukcji (Program Counter).

7. Tryby adresowania (operandy)

Tryb	Format	Obliczanie adresu
Natychmiastowy (Imm)	<code>\$Imm</code>	<code>Imm</code>
Rejestrowy (Reg)	<code>%Ra</code>	<code>%Ra</code>
Bezpośredni (Mem)	<code>Imm</code>	<code>MIImm1</code>
Pośredni (Mem)	<code>(%Rb)</code>	<code>MI%Rb1</code>
Z przesunięciem	<code>D(%Rb)</code>	<code>MI%Rb + D1</code>
Skalowany (Ind/scaled)	<code>D(%Rb, %Ri, S)</code>	<code>MI%Rb+%Ri*S+D1</code>
Skalowany (baseless)	<code>(, %Ri, S)</code>	<code>MI%Ri * S1</code>

Legenda: `Imm` / `D` to stała liczbowa, `%Ra` / `%Rb` to rejestr bazowy, `%Ri` to rejestr indeksowy, a `S` to skala (tylko: 1, 2, 4 lub 8).

8. Flagi stanu i sterowanie

ZF	Zero. Wynik to 0 (np. argumenty są równe).
SF	Sign. Wynik jest ujemny (MSB = 1).
CF	Carry. Przepełnienie dla liczb bez znaku (unsigned).
OF	Overflow. Przepełnienie dla liczb ze znakiem (signed).

Sufiksy warunkowe (dla `jX` , `setX` , `cmovX`)

Sufiks	Znaczenie
<code>e / z</code>	Equal / Zero (równe / wynik to 0)
<code>ne / nz</code>	Not Equal / Not Zero (nierówne)
<code>s</code>	Sign (wynik ujemny, <code>SF=1</code>)

Liczyby ze znakiem (signed)

<code>g / ge</code>	Greater / Greater or Equal
<code>l / le</code>	Less / Less or Equal

Liczyby bez znaku (unsigned)

<code>a / ae</code>	Above / Above or Equal (No Carry)
<code>b / be</code>	Below / Below or Equal (Carry)

Translacja struktur kontrolnych (C → ASM)

- if-else:** `jX` (skok) lub `cmovX` (liczy obie ścieżki, bez kary za predykcję). `cmovX` odpada przy drogich gałęziach, wyluskaniach (`*p`) i efektach ubocznych (`x++`).
- switch :** tablica skoków → czas $O(1)$. Skok pośredni: `jmp *.L4(%rdi,8)` . Brak `break` ⇒ **fall-through**.
- Pętla for :** zawsze redukowana do **while** :
`init; while(cond) { body; update; }`

Wzorce asemblerowe dla pętli:

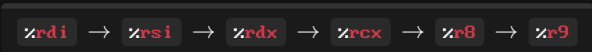
<code>do-while</code>	<code>while</code> (Jump-to-mid)	<code>while</code> (Guarded)
<code>loop:</code> Body if (T) goto loop	<code>goto test</code> <code>loop:</code> Body test: if (T) goto loop	<code>if (!T) goto done</code> <code>loop:</code> Body if (T) goto loop done:

9. Procedury i stos

Stos rośnie **w dół** (niższe adresy); `%rsp` wskazuje **wierzchołek** (najniższy zajęty adres). `push` zmniejsza `%rsp` o 8, `pop` zwiększa o 8.

- `call dest` : odkłada adres powrotu na stos i skacze do `dest` .
- `ret` : zdejmuje adres powrotu ze stosu i skacze pod niego.

10. Konwencja Wywoływań (System V ABI)



Argumenty 7+: Na stosie od końca. Wynik w `%rax` .

Rejestry caller-saved vs callee-saved

Caller-saved	Callee-saved
(Wołający musi zapisać) Mogą zostać nadpisane w funkoji. By je zachować, caller kładzie je na stos przed <code>call</code> .	(Wołany musi przywrócić) Muszą zachować stan. Callee musi zapisać je na stos i odtworzyć przed <code>ret</code> .
<code>%rax</code> , <code>%rdi</code> , <code>%rsi</code> , <code>%rdx</code> , <code>%rcx</code> , <code>%r8</code> - <code>%r11</code>	<code>%rbx</code> , <code>%rbp</code> , <code>%r12</code> - <code>%r15</code>

Ramka stosu

Każde wywołanie funkcji ma własną ramkę (stąd **rekurencja** działa naturalnie).

Ramka potrzebna, gdy: brak rejestrów na zmienne (spilling), lokalna tablica/struktura, lub pobrano adres zmiennej (&).

Prolog

```
pushq %rbp ;
zapisz stary base ptr
movq %rsp, %rbp ;
nowy base ptr = rsp
subq $32, %rsp ;
alokacja 32 bajtów
```

Epilog

```
addq $32, %rsp ;
zwolnij alokację
popq %rbp ;
przywróć base ptr
ret ;
skok powrotny
```

`leave` : zastępuje `movq %rbp, %rsp` + `popq %rbp` jedną instrukcją.

11. Katalog instrukcji (AT&T)

Opcode	Efekt	Opis
<code>mov S, D</code>	$D \leftarrow S$	Kopiuje wartość z S do D.
<code>lea S, D</code>	$D \leftarrow \text{addr}(S)$	Oblicza adres S (bez czytania pamięci).
<code>add S, D</code>	$D \leftarrow D + S$	Dodawanie (ustawia flagi).
<code>sub S, D</code>	$D \leftarrow D - S$	Odejmowanie (ustawia flagi).
<code>imul S, D</code>	$D \leftarrow D * S$	Mnożenie liczb ze znakiem.
<code>sal / shl k, D</code>	$D \ll= k$	Przesunięcie w lewo (mnożenie przez 2^k).
<code>sar k, D</code>	$D \gg= k$	Arytmetyczne w prawo (dzielenie). Kopiuje znak .
<code>shr k, D</code>	$D \gg= k$	Logiczne w prawo. Dopełnia zerami.
<code>and / or S, D</code>	$D \leftarrow D \& / S$	Operacje bitowe (ustawiają flagi).
<code>xor S, D</code>	$D \leftarrow D \wedge S$	Często jako <code>xor %rax, %rax</code> do zerowania.

Porównania i sterowanie

<code>cmp S1, S2</code>	$S2 - S1$	Ustawia flagi jak odejmowanie, nie zapisuje wyniku.
<code>test S1, S2</code>	$S2 \& S1$	Ustawia flagi jak AND .
<code>jX dest</code>	$\text{if}(X) \%rip \leftarrow \text{dest}$	Skok warunkowy (X = warunek).
<code>setX D</code>	$\text{if}(X) D \leftarrow 1$	Ustawia najmłodszy bajt na 0 lub 1 wg flag.
<code>cmovX S, D</code>	$\text{if}(X) D \leftarrow S$	Warunkowe kopiowanie (zamiast skoku).

Stos i zarządzanie procedurami

<code>push S</code>	$\%rsp - = 8$ $M[\%rsp] \leftarrow S$	Odkłada na stos (zmniejsza <code>%rsp</code>).
<code>pop D</code>	$D \leftarrow M[\%rsp]$ $\%rsp + = 8$	Zdejmuje ze stosu do D (zwiększa <code>%rsp</code>).
<code>call dest</code>	<code>push %rip</code> $\%rip \leftarrow \text{dest}$	Skok do funkcji (zapisuje adres powrotu na stosie).
<code>ret</code>	<code>pop %rip</code>	Zdejmuje adres powrotu ze stosu i skacze pod niego.
<code>leave</code>	$\%rsp \leftarrow \%rbp$ <code>pop %rbp</code>	Sprząta ramkę stosu (odwrotność prologu).

Uwaga o sufiksach: Instrukcje przyjmują sufiks określający rozmiar danych: `b` (8-bit), `w` (16-bit), `l` (32-bit), `q` (64-bit).
Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).